Short Paper: Gnomon: A Streaming Storage Engine for Temporal Graphs

Anonymous submission

Abstract

Temporal graphs whose edges are immutable timestamped events (audit streams, agent execution traces, interaction logs) are ingested as never-ending, near-sorted streams; queried along time and causality; and retained under time-based policies. No deployed storage layer serves this combination: entity-keyed graph stores demote the timestamp to a payload attribute, columnar and LSM engines pay a join per traversal hop, versioned temporal graph stores assume few property-rich versions per entity, and statically time-sorted layouts answer queries well while being unable to ingest at all.

Gnomon keys storage on time itself: each timestamp maps to a virtual address by a constant-time shift, and demand paging absorbs temporal sparsity, so no index is built or maintained. Ingestion becomes a lock-free $O(1)$ append independent of arrival order; retention becomes a page release over an address prefix; compaction is explicit and off the ingest path; queries remain correct *during* concurrent ingestion through a per-slot publication protocol; and because engine state is a pure function of the edge multiset, recovery is re-ingestion. We evaluate Gnomon against a columnar engine, three CSR layouts, and the most recent versioned temporal graph store, with element-wise answer verification, on fourteen datasets including a billion-edge enterprise corpus, and show that keying storage on time wins the streaming and traversal axes by orders of magnitude while conceding bounded, measured costs on the structural axis.

1 Introduction

A growing class of graphs is built from *events*. A system call touches a file; an agent invokes a tool; a host authenticates to another; a user replies to a message. Each edge carries the moment it happened, never changes afterwards, and arrives as part of a stream that does not end. For a storage system this combination is unusual and demanding: writes are append-only and unbounded, reads are constrained by time and causality rather than by key, and the retention policy, keeping the last N days, is itself a predicate on the write key. A single endpoint under ordinary load emits tens of thousands of such events per second, and a fleet multiplies that, so what the layout does with the time dimension determines the write path, the read path, the eviction path, and the recovery path at once.

Existing storage engines all approach this workload with a layout designed for something else. Dynamic graph stores [2, 3, 11, 14, 21, 26, 29, 32] key storage on source-vertex identity and treat the timestamp as payload. A time-constrained query then becomes a structural scan with a per-edge filter, and a retention pass has to find expired edges wherever vertex order scattered them. Columnar and LSM engines [24] have the opposite problem: ordered by time, they answer range filters well, but every hop of a traversal is a fresh join against the edge table. Versioned temporal graph databases [8, 16] solve a different problem, retrieving the valid *version* of an updated entity, and their structures assume few, property-rich versions per entity, which an event stream violates by construction. A statically time-sorted CSR answers temporal queries close to optimally, but it is not a storage engine: its only update path is a full rebuild, and during that rebuild it is blind.

Gnomon keys storage on time. The starting observation is that a 64-bit virtual address space is orders of magnitude larger than the time range of any recorded workload, so instead of maintaining an index from timestamp to location, we let the timestamp *be* the location:

$$A(T) \;=\; A_{\text{base}} + \big((T - T_0) \gg S\big) \times P, \tag{1}$$

one subtraction and one shift, with the operating system's demand paging absorbing temporal sparsity at no data-structure cost.

What follows from that layout is the point of the paper. An edge's home is computed, not searched, so ingestion is a wait-free slot claim; the computation ignores arrival order, so late events cost nothing extra; retention is a monotone predicate on time, so expiry releases an address prefix; and noth-

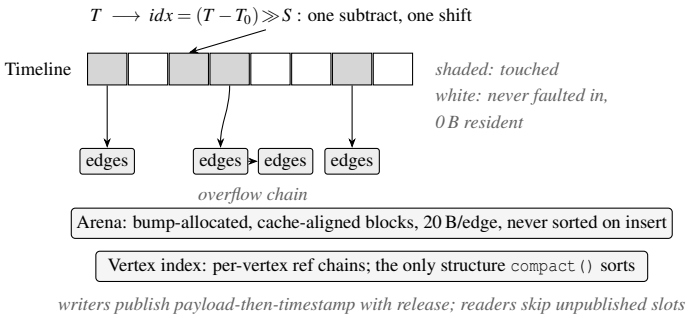


Figure 1: Gnomon's layout. A timestamp *computes* its timeline slot, so ingestion never searches and never sorts, and temporal sparsity is absorbed by the MMU rather than by a structure the engine maintains. Entity-keyed access lives in a separate index that compaction, never insertion, sorts.

ing is updated in place, so recovery is replaying an append-only log through the normal insert path. None of these is engineered on top of the layout; each is a consequence of it.

Time-ordering itself is not new. Log-structured file systems [25], LSM trees [20] and time-series stores [23] have organised writes by time for decades, but in all of them an auxiliary structure still maps keys to the data: sorted runs and a manifest, or per-series chunk indexes. Gnomon removes that level. Because the key *is* the time and the time *is* the address, there is no run, manifest or chunk index to maintain, merge or recover, which is why compaction here is optional rather than structural and recovery is replay rather than log analysis [19].

The design also has costs, and a storage paper owes their measurement as much as their mitigation. A layout that refuses to organize by entity must pay something on entity-keyed operations, and one that never sorts during ingestion must either scan unsorted neighborhoods or sort them later. We measure both and report the operations where a competing design beats us outright (Section 4).

We contribute: (i) a storage design that keys temporal graphs on time, with proofs for its addressing, space, granularity, and concurrency properties (Section 2); (ii) a publication protocol under which readers never observe a torn or unpublished edge during lock-free multi-writer ingestion (Theorem 4); and (iii) an evaluation against two streaming graph stores, a columnar engine and three CSR variants that verifies element-wise answer equality before reporting any latency, and that evaluates recovery rather than merely asserting it (Section 4).

2 Design

2.1 Time-keyed addressing

Equation (1) maps a timestamp to a slot in one anonymous memory mapping (Figure 1). Two properties turn that arithmetic into a storage layout.

Theorem 1 (Addressing)**.** *For any timestamp t, the slot holding every edge stamped t is computed in $O(1)$ worst case, with no data-dependent memory access.*

Proof. The map is one subtraction and one shift. Two timestamps land in the same slot exactly when $\lfloor (t - T_0)/2^S \rfloor = \lfloor (t' - T_0)/2^S \rfloor$, i.e. when they share a window, so every edge stamped t is registered under that one index and no other slot can hold one. No search structure is consulted, so the bound is unconditional rather than expected-case. A B-tree or LSM index gives $O(\log E)$ with pointer chases; a hash index gives *expected* $O(1)$ with worst-case collisions. □

Theorem 2 (Space)**.** *Let P be the page size and W_{occ} the number of non-empty windows. Resident timeline memory is at most $W_{occ} \cdot P$ plus page-table overhead the kernel materializes lazily; untouched windows cost zero bytes, whatever the virtual reservation.*

Proof. The timeline is an anonymous mapping of pointer slots. A page becomes resident only when first written, and a slot is written only when its window receives its first edge. Resident pages are therefore at most the number of pages holding at least one occupied slot, which is at most W_{occ} since each occupied slot dirties one page. Untouched ranges are never faulted in and never appear in the page table. □

Sparsity is absorbed by the MMU, not by a structure the engine maintains. The mapping is *anonymous*, so the failure modes catalogued against mmap in databases [1], namely torn file pages, SIGBUS on truncation and eviction stalls, do not apply: we use mmap as an allocator reserves address space, not as a buffer pool.

The shift S sets how much time one slot covers. Too coarse and windows overfill; too fine and the timeline sprawls. Gnomon picks S from stream statistics, and the choice is bounded.

Theorem 3 (Granularity)**.** *For $E \geq 2\tau$ events with target occupancy τ, the selected shift yields mean window occupancy $\mu \in [\tau/2, 4\tau)$.*

Proof sketch. The selector takes the smallest S with $2^S \geq \lfloor L/D \rfloor$ for $D = \lfloor E/\tau \rfloor$ desired windows, so the realized count $W = \lceil L/2^S \rceil$ obeys $D/2 < W \leq D+1$ and $\mu = E/W$ falls in the stated interval. The artifact carries the full derivation. □

Algorithm 1: Adaptive granularity selection. Runs once on a stream prefix; re-running on a later prefix changes only future windows, since Equation (1) is evaluated per edge at insert time.

Input: event count E, span L, target occupancy τ
Output: shift S

$D \leftarrow \max(1, \lfloor E/\tau \rfloor)$ // windows we want
$w \leftarrow \max(1, \lfloor L/D \rfloor)$ // width we want
$S \leftarrow \lceil \log_2 w \rceil$ // width we can address
return S

The two factors of two are real, not slack: one from rounding the window width up to a power of two, which shift addressing cannot avoid, one from the ceiling on the window count. Across our corpora a single target yields per-window densities from 32 to 32,768 edges, and the bound holds at both ends.

2.2 Lock-free ingestion, correct concurrent reads

An insert computes its slot (Theorem 1), claims an index in the window's block with one `fetch_add`, and installs overflow blocks by CAS. Every failed CAS means some other insert succeeded, so ingestion is lock-free and the slot claim itself is wait-free. Nothing sorts and nothing rebalances, so arrival order does not matter. That is what separates Gnomon from every sorted-on-ingest design as soon as events arrive late or out of order.

Readers are protected by a publication protocol instead of locks. A writer fills an edge's payload first and stores its timestamp *last* with release semantics. Fresh arena pages are zero-filled, so a zero timestamp marks a slot that is not yet published; readers acquire-load the timestamp and skip those. Vertex-index references publish through their block pointer the same way. One extra ordering rule makes the fast read path safe as well: an append clears the bucket's *sorted* flag *before* it bumps the bucket counter.

Theorem 4 (Concurrent reads)**.** *Under this protocol no query observes a torn, partial, or unpublished edge during concurrent multi-writer ingestion, and a reader that finds the sorted flag set may binary-search the counted prefix.*

Proof. For the first claim: a reader only accepts a slot whose timestamp is non-zero, and the writer's release store of the timestamp happens after its payload writes, so an acquiring reader that sees the timestamp also sees the payload. A slot is written once and never mutated, so no torn state exists after publication.

For the second: a reader loads the counter (acquire) and then the flag. Suppose it observes the flag still set. Any append that could have extended the counted prefix cleared the flag before bumping the counter, and the reader's acquire

Algorithm 2: Publication protocol. The two release stores and the order of lines 3–4 are what Theorem 4 rests on: clearing *sorted* before bumping *count* is what lets a reader trust a binary search, and storing t last is what makes a half-written slot invisible.

1 APPEND(e)
2 $b \leftarrow$ block for slot $(e.t - T_0) \gg S$ // Theorem 1
3 $b.sorted \leftarrow \bot$ (release)
4 $i \leftarrow$ FETCHADD($b.count, 1$) (acq-rel)
5 **if** $i \geq b.cap$ **then** install overflow block by CAS; retry
6 write $e.src, e.dst, e.w$ into $b[i]$
7 $b[i].t \leftarrow e.t$ (release: publishes the slot)
8 EXPAND(v, t_{lo}, t_{hi})
9 **foreach** *bucket b of v* **do**
10 $n \leftarrow b.count$ (acquire)
11 **if** *b.sorted (acquire)* **then** binary-search $b[0..n)$
12 **else** scan $b[0..n)$, skipping $t = 0$

on the counter synchronizes with every bump it observes; so if any such bump were visible, the corresponding clear would be visible too and the flag would be unset. The observed prefix is therefore exactly the one compaction verified and published, and binary search over it is sound. □

2.3 The compaction contract

Appends leave a vertex's neighborhood in arrival order, so a time-windowed expansion has to scan it: $O(d)$. An explicit `compact()` pass sorts each neighborhood by timestamp once, and afterwards expansion binary-searches in $O(\log d + k)$ for k results. Compaction is deliberately *not* on the insert path: an early build that compacted after every batch ingested at 11K events/s, because each batch re-walked the whole index, which is the same $O(\text{index})$-per-batch cost a sorted structure pays by design.

Queries are correct in both states; compaction changes their cost, not their answers. This is the paper's central trade. A statically sorted structure gets its query speed by making sorting *mandatory* and pays with a rebuild window during which its answers are wrong (Section 4.5); Gnomon makes sorting *optional* and pays a bounded constant until it runs. The contract is: *always appendable, always correct including mid-ingest; fast expansion after a compaction you schedule when convenient.*

2.4 Retention and durability

Retention is a monotone predicate on time, so under Equation (1) it is an operation on an address prefix. `evict_before(t)` walks the timeline prefix, merges the block extents it finds, and returns whole pages to the OS with `madvise`; readers of other windows are untouched. There is

no scan of the edge set and no tombstone compaction, because expiry and address order coincide.

Durability uses a property most databases do not have: engine state is a pure function of the edge multiset. Edges are immutable, never updated, and deletion is retention policy rather than semantics. The log is therefore just framed, checksummed batches of raw edges; an asynchronous writer overlaps it with insertion; and recovery is replaying the log through the ordinary insert path. There is no undo phase, no redo phase, and no idempotence bookkeeping, because replaying a batch twice is indistinguishable from the multiset having contained it twice, which it cannot, since each batch is framed once. A torn tail frame fails its length or checksum test and is discarded.

3 Implementation

Gnomon is ∼6,500 lines of dependency-free C++17: the engine, nine graph algorithms, converters for two audit-log formats, and seven correctness suites, among them a multi-writer/multi-reader stress test of Theorem 4. Interior windows of a range scan are copied without any timestamp comparison, since only boundary windows can contain out-of-range edges, and boundary scans are AVX2-vectorized. Source, converters, and every experiment driver will be released.

4 Evaluation

Setup. 20-core Xeon, 188 GB RAM, Ubuntu 20.04. Fourteen corpora spanning five orders of magnitude (Table 1): LANL enterprise authentication [9] (1.05B events, 17,666 hosts, 58 days), DARPA TC whole-system provenance (CADETS 22.5M and THEIA 97.3M information-flow edges), and 12 public temporal graphs [15] from 60K to 113M edges. Baselines, all driven in-process: GraphOne [11] and LiveGraph [32] built from source; DuckDB v1.5.5 through its C API; a time-sorted CSR (`tcsr`, the strongest static competitor), an arrival-order entity CSR (`ucsr`), and a topology-only CSR; and TVA [16] via its released artifact. Before any latency is reported we check that the engines returned the same answer element by element. Latencies are heavy-tailed, so we report medians with tails or results binned by answer size, never means.

4.1 Ingestion

Across the 13 corpora of Table 1, Gnomon ingests at 7.0–94.6M events/s on eight threads, the one optional compaction pass costing at most 3.8 s on the largest (113M edges). Rate tracks arrival density rather than corpus size: the two audit-log corpora, whose events crowd into days rather than years, are the fastest.

Table 1: Breadth across 13 corpora spanning four orders of magnitude. Ingestion is what the stream sees; compaction is the one optional pass. Traversal is the median time-respecting forward traversal over a 1% window. Bytes/edge are resident, against a 20 B packed edge. Eight threads.

Corpus	Edges	$\|V\|$	Span (d)	Ingest (Meps)	Comp. (s)	Trav. (μs)	B/ edge
CollegeMsg	60K	2K	194	5.2	0.00	1.0	158
email-Eu	332K	1K	804	6.8	0.01	1.0	79
act-mooc	412K	7K	30	9.0	0.01	0.5	87
mathoverflow	507K	25K	2350	7.5	0.02	0.5	87
askubuntu	964K	159K	2614	6.8	0.04	0.5	91
superuser	1.4M	194K	2773	7.0	0.06	0.5	88
wiki-talk	7.8M	1.1M	2320	6.2	0.34	0.5	68
cit-Patents	16.5M	6.0M	1826	7.4	0.76	0.5	77
TC-CADETS	22.5M	2.9M	11	5.8	0.86	0.5	65
stackoverflow	63.5M	2.6M	2774	5.8	3.11	0.5	66
LiveJournal	69.0M	4.8M	1157	5.3	3.50	0.7	71
TC-THEIA	97.3M	6.9M	10	4.9	3.45	0.5	63
bitcoin	113.1M	48.1M	2585	1.9	5.96	0.5	107
plus LANL, 1.05B edges (§4.1, §4.5)							

The billion-edge LANL corpus is absorbed at 77.5M events/s, and the rate does not decay as history accumulates: streaming 256M of its events in sixteen equal chunks holds 51.2–56.5M from the first chunk to the last, memory linear in edges, no event refused.

Readers cost the writer less than durability does. With four issuing validated windowed queries throughout, ingestion falls from 47.9 to 36.6M events/s on wiki-talk and 43.5 to 29.7M on stack-overflow, and across three corpora every query over a window the stream had already passed returned the answer a quiescent engine gives, on every one of hundreds of thousands of repetitions. Forcing the log to stable storage costs far more: the same stream absorbs 57.4M with no log and 28.8M with a log written but never flushed, which is the contract GraphOne's log implements, against 6.5M under 64 MB group commit and 1.1M with an fsync per batch.

A versioned temporal store is the wrong shape for this load rather than merely slower: driving TVA's artifact with the same stream did not finish wiki-talk's 7.8M events in 90 minutes, at 41.6 GB RSS (Figure 2a), because every repeated (src, dst) contact extends one edge's version chain and inverts its assumption of few, property-rich versions per entity.

4.2 Time-respecting traversal

On multi-hop time-respecting traversal run to fixpoint, Gnomon leads DuckDB by 150–700× across all answer sizes; at $|R| > 1{,}000$ the median is 12 ms against 2.43 s (Figure 3a). The reason is structural: each hop costs a relational engine a join against the edge table. The time-sorted CSR stays 2.7× faster than Gnomon at high degree, which is the measured price of refusing to sort during ingestion; Section 4.5 prices what the CSR pays for that privilege.

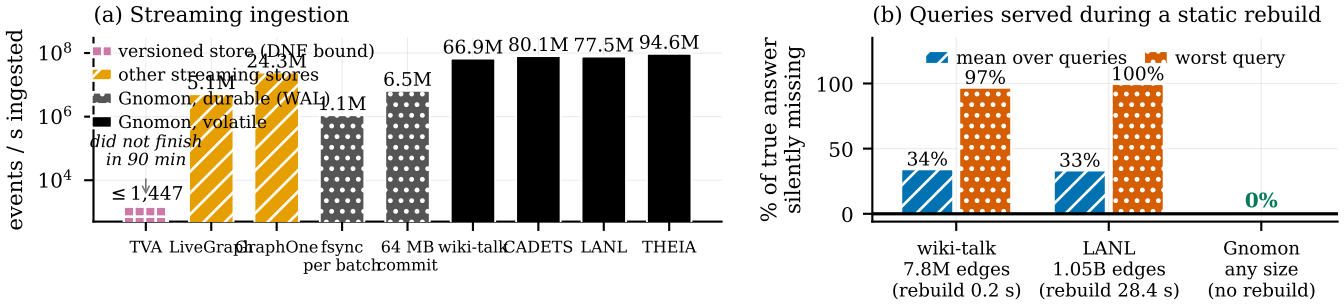


Figure 2: The dynamic axis. **(a)** Ingestion on eight threads, log scale; the three baseline bars are wiki-talk, so they are like for like. The versioned store did not finish wiki-talk's 7.8M events within a 90-minute cutoff, so its bar is an upper bound, and a statically sorted CSR has no ingest path to place on this axis at all. Gnomon's durable modes are bounded by fsync, not by the engine. **(b)** Queries answered from a static structure's pre-rebuild snapshot are not stale but silently incomplete; Gnomon never has a rebuild window, at any corpus size.

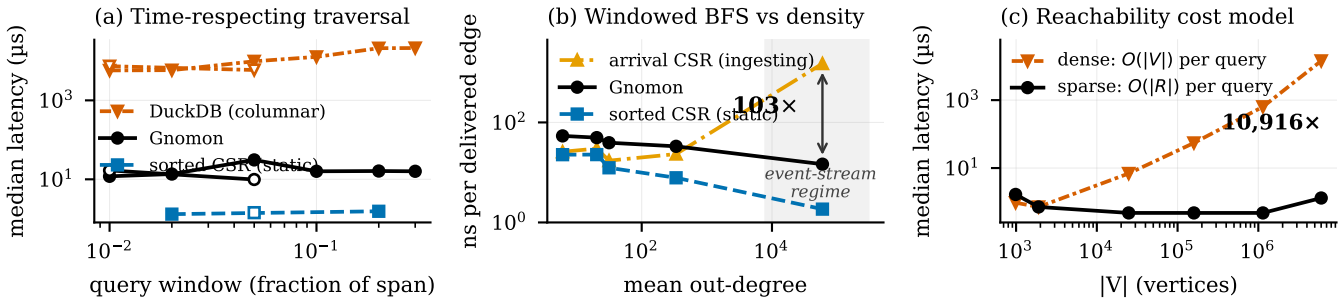


Figure 3: The time axis, three ways. **(a)** Time-respecting traversal to fixpoint across query windows on two corpora (filled markers: sx-mathoverflow; open: wiki-talk); answers verified element-wise identical across engines before timing. **(b)** Windowed BFS, normalised to cost *per delivered edge*, the only size-independent comparison across corpora spanning 60K to 1.05B edges. An arrival-order adjacency must scan an entire neighborhood to find the in-window edges, so its per-edge cost explodes with density (26 → 1500 ns); Gnomon's compacted buckets amortise a binary search, so its cost *improves* (54 → 15 ns). The crossover lands inside the regime event streams occupy. **(c)** Sparse temporal reachability makes query cost track the answer rather than the graph.

Against an *ingesting* entity-keyed layout the ordering flips with density (Figure 3b): its per-delivered-edge cost rises from 26 ns at degree 6 to 1,500 ns at degree 59,517 because it scans whole neighborhoods, while Gnomon's compacted buckets amortise a binary search and *fall* from 54 to 15 ns. The crossover sits near degree 100, inside the range event streams occupy.

Two more results bound the picture. On time-*range* filtering, DuckDB's home ground, Gnomon's vectorized scan wins the median 10.6× (77 μs against 819 μs) with p95 at parity. And on structure-only algorithms, decomposing each run into the share spent delivering adjacency under checksum-verified executions, Gnomon delivers at 0.9–5.8× the topology-only CSR optimum over 29 measurements on 11 corpora, median 2.1×. The spread separates by access pattern: frontier-driven traversal stays at 1.4–2.2× regardless of corpus size, including on the billion-edge corpus, while whole-graph iteration walks every bucket chain on every round where a CSR walks one contiguous array, reaching 5.8× at 69M edges. This is the concession: the price of not organising by entity, paid for an ingest path the optimum does not have.

Gnomon's weakest operation is the full-graph as-of snapshot, answered by enumerating and reducing: 218 s on CADETS. That is what TVA's version-skipping is built for, and we report it as its win.

4.3 In-category baselines

The comparison a storage reader asks for is against stores built for the same job. We build two from source and drive all three through one query list, comparing a checksum over the delivered (dst, t) pairs so that a faster number cannot be a different question: 54 of 54 LiveGraph configurations and 9 of 9 GraphOne corpora agree.

Against LiveGraph [32], ingestion is 2.7–15.7× faster at

roughly half the resident set, and windowed neighbour delivery 1.6–194×. Against GraphOne [11] the ingest comparison is close, 0.92–2.76× measured like for like against the point its archiving makes edges visible, and the separation is in the query: 1.4–16.3×, because its adjacency carries no order on time and selecting a window costs it a read amplification of 8–84× (4.3B edges scanned to deliver 51M on CADETS). Uncompacted we are within 1.3–1.7× of GraphOne's scan rather than ahead of it; the separation is what compaction buys.

Run end to end, extracting a window, computing over it with identical code, and writing results back, extraction on a 63.5M-edge corpus runs 70× ahead of LiveGraph at a 0.01% window and exactly level at 100%: its cost barely moves across a 2000× change in result size. A columnar engine indexed on time is the closer competitor, 1.2–1.9× behind at narrow windows.

4.4 Recovery

Recovery is re-ingestion (Section 2.4), which makes it fast and the claim checkable. Replaying logs from 1M to 1.024B edges the rate is flat at 16–35M events/s, and the billion-edge LANL corpus is back in 29.6 s. Killing a live writer with `SIGKILL` at an arbitrary instant, 50 times over two corpora, the replayed engine is every time byte-identical to the input prefix and query-equivalent to an engine built from exactly those edges. A signal cannot produce a torn frame; truncating the log at a random byte or flipping one bit inside a frame can, and across 60 such injections replay stopped at the last intact frame and never admitted a corrupt batch.

4.5 Rebuild versus stream

The time-sorted CSR's only update path is a full rebuild, 28.4 s on LANL, and a query served from the pre-rebuild snapshot is not stale but silently *wrong*: 68 of 100 returned incomplete answers, losing 33.2% on average and 99.5% at worst (Figure 2b). Gnomon has no blind window. Its own compaction costs about twice that rebuild, 55.7 s, but the two differ in kind: the rebuild is on the correctness path, compaction on the performance path. That is the contract.

4.6 Retention and a real workload

Under a stream with a retention horizon, memory reaches a steady state rather than growing: 7.8M events pass through while resident memory holds at 250–310 MB, each eviction costing 0.3–0.4 ms and leaving the ingest rate undisturbed. Against a columnar `DELETE`, releasing 90% of a 16.5M-edge history is 5–29× faster; both reclaim fully, so the win is speed. Tracing all 749 labelled red-team alerts of LANL to causal fixpoint takes 46.5 s in total (median 24.6 μs, p95 353 ms), and on DARPA CADETS a forward trace reconstructs the published attack chain in causal order inside an impact set 0.0093% of the graph, at 133 μs.

Negative results, kept. Backing the dense arenas with 2 MB pages was implemented and measured and loses: on 240M LANL events it costs 28% of ingest throughput and 69% more resident memory, so it ships disabled. Binary search into a sorted bucket likewise loses below roughly 32 refs. And this original retention hypothesis, that the columnar engine would mark deleted rows without reclaiming, was false.

5 Related Work

Versioned temporal graph stores. TVA [16], AeonG [8] and Aion [28] organize versions of *updated entities* for snapshot retrieval. Delta encoding and version-chain locality, their central difficulties, do not arise in an append-only stream, and time-respecting reachability is not expressible over a snapshot surface. Aion comes closest, indexing updates by time, but it indexes *revisions* of entities rather than making time the address of the events themselves. **Structural dynamic graph stores** [2,3,14,21,26,29,32] optimize the entity axis and treat time as payload; none answers time-respecting queries natively, and Section 4.3 prices what that costs. RapidStore [4] also decouples version data from graph data, but at subgraph granularity, still keyed on vertex id, with versions reclaimed rather than retained. GraphOne [11] is the closest prior work at this venue: a dual-format store keeping a recent edge log beside a compacted adjacency, with a marker bit written after the payload so readers detect a partial update, the same discipline Theorem 4 formalises. But its edge log is a staging area that must drain into the entity-keyed format, and it states it is not designed for arbitrary historic views; here the time-ordered form *is* the durable form. LLAMA [17] materialises versions. **Temporal queries** such as time-respecting reachability are classical [7, 30]; we adopt their semantics and supply the layer below the frameworks [12, 27]. **Concurrency.** The append path follows the standard lock-free discipline [18]; Theorem 4 is a linearizability argument [6]. **Columnar/LSM engines** [24] own predictable range-filter tails and lose traversal by orders of magnitude; the case against mmap [1] targets file-backed buffer management, which an anonymous reservation sidesteps. **Provenance systems** [5, 10, 22, 31] motivate the rates; their reduction techniques [13] exist because no storage layer made retention affordable.

6 Conclusion

Keying storage on time collapses the machinery an event stream needs: the index becomes a shift, ingestion an append, retention a page release, and recovery a replay.

References

[1] Andrew Crotty, Viktor Leis, and Andrew Pavlo. Are you sure you want to use MMAP in your database management system? In *Proc. CIDR*, 2022.

[2] Laxman Dhulipala, Guy E. Blelloch, and Julian Shun. Low-latency graph streaming using compressed purely-functional trees. In *PLDI*, 2019.

[3] Per Fuchs, Domagoj Margan, and Jana Giceva. Sortledton: A universal graph data structure. *ACM SIGMOD Record*, 2023.

[4] Chiyu Hao, Jixian Su, Shixuan Sun, Hao Zhang, Sen Gao, Jianwen Zhao, Chenyi Zhang, Jieru Zhao, Chen Chen, and Minyi Guo. Rapidstore: An efficient dynamic graph storage system for concurrent queries. *PVLDB*, 18(10):3587–3600, 2025.

[5] Wajih Ul Hassan, Mohammad A. Noureddine, Pubali Datta, and Adam Bates. OmegaLog: High-fidelity attack investigation via transparent multi-layer log analysis. In *Proc. NDSS*, 2020.

[6] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3), 1990.

[7] Petter Holme and Jari Saramäki. Temporal networks. *Physics Reports*, 519(3), 2012.

[8] Jiamin Hou, Zhanhao Zhao, Zhouyu Wang, Wei Lu, Guodong Jin, Dong Wen, and Xiaoyong Du. Aeong: An efficient built-in temporal support in graph databases. *PVLDB*, 17(6), 2024.

[9] Alexander D. Kent. Comprehensive, Multi-Source Cyber-Security Events. Los Alamos National Laboratory, 2015.

[10] Samuel T. King and Peter M. Chen. Backtracking intrusions. In *Proc. SOSP*, 2003.

[11] Pradeep Kumar and H. Howie Huang. GraphOne: A data store for real-time analytics on evolving graphs. In *Proc. USENIX FAST*, 2019.

[12] Aapo Kyrola, Guy Blelloch, and Carlos Guestrin. GraphChi: Large-scale graph computation on just a PC. In *Proc. USENIX OSDI*, 2012.

[13] Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. LogGC: Garbage collecting audit log. In *Proc. ACM CCS*, 2013.

[14] Dean De Leo and Peter Boncz. Teseo and the analysis of structural dynamic graphs. In *PVLDB*, volume 14, 2021.

[15] Jure Leskovec and Andrej Krevl. SNAP datasets: Stanford large network dataset collection. http://snap.stanford.edu/data, 2014.

[16] Wenhao Li, Zhanhao Zhao, Jinhao Dong, Jiamin Hou, Wei Lu, Yunhai Wang, and Xiaoyong Du. TVA: A version-aware temporal graph storage system for real-time analytics. In *PVLDB*, volume 19, 2026.

[17] Peter Macko, Virendra J. Marathe, Daniel W. Margo, and Margo I. Seltzer. LLAMA: Efficient graph analytics using large multiversioned arrays. In *Proc. IEEE ICDE*, 2015.

[18] Maged M. Michael and Michael L. Scott. Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In *Proc. ACM PODC*, 1996.

[19] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Trans. Database Syst.*, 17(1), 1992.

[20] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. The log-structured merge-tree (LSM-tree). *Acta Informatica*, 33(4), 1996.

[21] Prashant Pandey, Brian Wheatman, Helen Xu, and Aydin Buluc. Terrace: A hierarchical graph container for skewed dynamic graphs. In *Proc. ACM SIGMOD*, 2021.

[22] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eyers, Margo Seltzer, and Jean Bacon. Practical whole-system provenance capture. In *Proc. SoCC*, 2017.

[23] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang, Justin Meza, and Kaushik Veeraraghavan. Gorilla: A fast, scalable, in-memory time series database. *PVLDB*, 8(12), 2015.

[24] Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proc. SIGMOD*, 2019.

[25] Mendel Rosenblum and John K. Ousterhout. The design and implementation of a log-structured file system. In *Proc. ACM SOSP*, 1991.

[26] Yue Shen, Jie Zhang, Huawei Cao, Yuan Zhang, and Xuejun An. Dcsr: A fast data structure with leaf-oriented locks for streaming graph processing. In *EDBT*, 2026.

[27] Julian Shun and Guy E. Blelloch. Ligra: A lightweight graph processing framework for shared memory. In *Proc. ACM PPoPP*, 2013.

[28] Georgios Theodorakis, James Clarkson, and Jim Webber. Aion: Efficient temporal graph data management. In *EDBT*, volume 27, pages 501–514, 2024.

[29] Brian Wheatman and Helen Xu. A parallel packed memory array to store dynamic graphs. In *ALENEX*, 2021.

[30] Huanhuan Wu, James Cheng, Silu Huang, Yiping Ke, Yi Lu, and Yanyan Xu. Path problems in temporal graphs. *PVLDB*, 7(9), 2014.

[31] Bo Zhang et al. TAPAS: An efficient online APT detection with task-guided process provenance graph segmentation and analysis. In *Proc. USENIX Security*, 2025.

[32] Xiaowei Zhu, Guanyu Feng, Marco Serafini, Xiaosong Ma, Jiping Yu, Lei Xie, Ashraf Aboulnaga, and Wenguang Chen. LiveGraph: A transactional graph storage system with purely sequential adjacency list scans. *PVLDB*, 13(7), 2020.